When Self-Evolving Agents Learn Too Soon

Lazy Distillation Breaks Confirmation-Bias Loops and Preserves Workflow Patterns

Anonymous preliminary report

July 2026

Abstract

Self-evolving agents train on experience produced by their own current policy. Over many train–collect loops, this creates two coupled failures: a wrong action can become its own training target (*confirmation bias*), and learning the latest task mixture can erase previously successful workflows (*pattern loss*). Existing confidence gates, teacher averaging, and replay control target quality or memory, but they do not remove the immediate policy-update→next-data feedback edge. We propose LazyDistill, a change in learning time: keep the collector frozen for several non-IID waves, then train once on their pooled evidence. We construct a controlled benchmark of 24 confusable tool-use workflows and expose agents for up to 24 repeated self-evolution cycles. Simulator oracle actions let us directly measure the fraction of wrong pseudo-targets, while peak-to-final per-workflow success measures pattern loss. At 16 steps and 24 cycles, eager and SDAR-style gated distillation obtain 0.0% and 3.3% success; LazyDistill obtains $79.6\% \pm 0.8\%$, reduces pseudo-label error from 95.3% (eager) to 29.9%, and limits pattern loss to 6.4%. The evidence is controlled rather than a full LLM-agent reproduction, but it identifies update timing as a causal axis that existing self-distillation objectives leave open.

Paper at a glance

Question	Why do agents that repeatedly learn from their own trajectories collapse even when each individual update appears reasonable?
Diagnosis	Eager updates couple two failures: wrong actions become future targets (*confirmation bias*), while the latest non-IID wave overwrites previously solved workflows (*pattern loss*).
Intervention	Freeze the collector across several waves, pool their evidence, and update once. This removes within-window error feedback and makes common/rare patterns co-present during optimization.
Headline evidence	At 16 steps and 24 cycles, Eager reaches 0.0% success and SDAR-style gating reaches 3.3%; Lazy retains $79.6\% \pm 0.8\%$ with 6.4% pattern loss.

1 Introduction

Self-evolution asks an agent to turn its own interaction history into better future behavior. A current policy collects multi-step trajectories, converts them into pseudo-supervision using feedback or hindsight, updates its parameters, and repeats. This paradigm underlies agent self-training methods such as ReST-meets-ReAct and Re-ReST [Aksitov et al., 2023, Dou et al., 2024], and recent agentic RL methods such as SDAR, OPID, and SEED [Lu et al., 2026, Yang et al., 2026, Wu et al., 2026].

The long-horizon setting is different from ordinary one-shot self-distillation because the learner also controls its next training distribution. A genuinely self-evolving agent runs

$$\pi_k \longrightarrow \mathcal{D}_k(\pi_k) \longrightarrow \widehat{\mathcal{D}}_k \longrightarrow \pi_{k+1}, \tag{1}$$

where the current policy π_k determines the trajectory distribution $\mathcal{D}_k$, and therefore determines its own future supervision.

This closed loop couples two known learning problems into one agent failure. Pseudo-labeling research calls the first problem *confirmation bias*: an incorrect model prediction is reused as a label and becomes more confident [Arazo et al., 2020, Rodemann et al., 2023]. Continual-learning research calls the second broad problem *catastrophic forgetting*: later non-IID updates damage older capabilities [Luo et al., 2023]. We use *pattern loss* for a precise

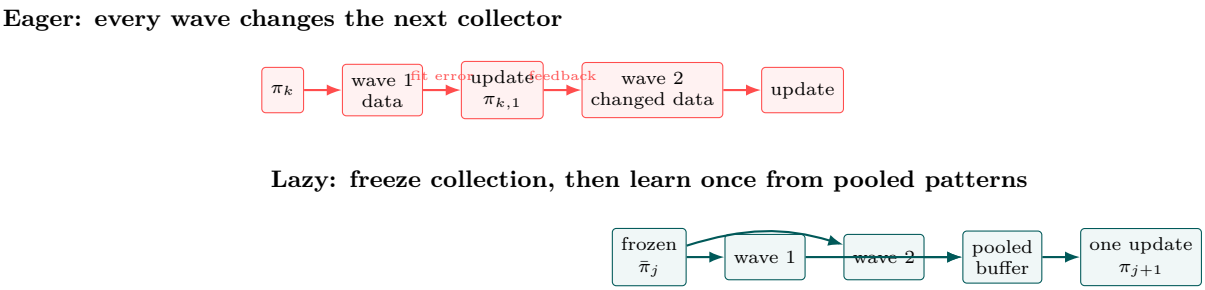


Figure 1: The intervention changes when learning affects future data. Eager updates create a short feedback loop; Lazy freezes the collector so errors cannot alter later waves inside the window, then jointly optimizes all observed patterns.

agent-level measurement of that second problem: how much full-workflow success falls from its best observed value to its final value. Neither term is claimed as new; the new question is why they reinforce each other over many policy-controlled collection loops.

Existing defenses leave the temporal feedback edge intact. Confidence filtering rejects low-confidence labels but fails when the agent is confidently wrong. Mean Teacher smooths targets but still updates after each non-IID wave. Replay retains old examples but may retain old mistakes. SDAR's positive-gap gate improves dense token supervision within agentic RL [Lu et al., 2026], yet it also controls loss weight rather than the time at which a learned policy changes future collection. Concurrent evidence indeed finds progressive collapse under multi-iteration agent experience internalization and favors stable off-policy signals [Chen et al., 2026].

Our solution is to make learning *lazy*, not weak. As Figure 1 illustrates, LAZYDISTILL freezes the collector across a window of different workflow waves and performs one update on their union. Freezing blocks an early mistake from changing later data in the same window; pooling makes old, new, common, and rare patterns compete in the same optimization event. The method therefore targets the causes of both failures without requiring an oracle at training time.

Our controlled study makes three contributions. First, it defines agent-observable probes that separately measure wrong-target feedback and workflow erasure. Second, it provides a procedural long-horizon benchmark in which the oracle is used only for diagnosis, not training. Third, across generic defenses and an SDAR-style gate, it shows that update timing is the only tested intervention that simultaneously maintains task success, low pseudo-label error, and low pattern loss.

2 Related Work and the Missing Axis

Confirmation bias in self-training. Confirmation bias was explicitly studied in deep pseudo-labeling by Arazo et al. [2020]: incorrect predictions can be selected as labels and reinforced. Confidence selection and Bayesian selection reduce exposure to uncertain targets [Radhakrishnan et al., 2023, Rodemann et al., 2023]. These methods address which examples enter an update. In our setting, however, an error can become highly confident and the update changes which states the agent visits next; example filtering does not break that temporal edge.

Forgetting under sequential learning. Catastrophic forgetting is the degradation of earlier capabilities while learning later tasks, and it has been observed during continual LLM fine-tuning [Luo et al., 2023]. Replay and teacher-based consistency are standard defenses [Tarvainen and Valpola, 2017, Yoo et al., 2024], while recent self-distillation fine-tuning reduces forgetting when learning from demonstrations [Shenfeld et al., 2026]. These methods preserve parameters or examples. Pattern loss instead tests an agent-specific outcome—whether a previously solved multi-step workflow is no longer solved after later self-generated updates.

Self-improving and self-evolving agents. STaR, ReST, ReST-meets-ReAct, and Re-ReST train from generated or repaired reasoning traces [Zelikman et al., 2022, Gulcehre et al., 2023, Aksitov et al., 2023, Dou et al., 2024]. SDAR uses a privileged-context teacher and a detached positive-gap gate; OPID and SEED distill hindsight skills from on-policy trajectories [Lu et al., 2026, Yang et al., 2026, Wu et al., 2026]. These papers improve supervision within an optimization round, but do not isolate the causal effect of update frequency across many non-IID rounds. The closest concurrent work reports multi-iteration capability collapse [Chen et al., 2026]; our distinction is a controlled intervention on update timing plus separate measurements of error reinforcement and workflow erasure.

3 Failure Setting

The failure can be understood through one concrete tool-use example. Consider two visually similar workflows, one requiring `search→open→extract` and a rarer one requiring `search→filter→extract`. If an eager agent mistakes `open` for `filter`, its own pseudo-target may approve `open`; the update then makes `open` more likely in later collections. As common examples dominate later waves, the rare `filter` workflow can fall from previously solved to consistently failed. Confirmation bias describes the wrong target; pattern loss describes the lost end-to-end workflow.

Running example: one error, two long-horizon consequences

Time	Event in the self-evolution loop	Diagnostic consequence
k	Agent chooses wrong `open`; its pseudo-target copies `open`.	A self-reinforced error enters training.
$k+1$	The updated policy visits states caused by `open` more often.	Confirmation bias grows across collection rounds.
$k+m$	Common `open` workflows dominate; rare `filter` examples are overwritten.	Previously acquired workflow success is lost.

An episode is a partially observed workflow with horizon H. At state s_t, the policy selects one of eight tools. The hidden workflow program defines the oracle action $a_t^\star$. A weak hindsight verifier and the policy distribution form a pseudo-label:

$$q_k(a \mid s) = (1-\lambda)\,\mathrm{Sharpen}(\pi_k(a \mid s)) + \lambda v(a \mid s), \qquad \hat{a} = \arg\max_a q_k(a \mid s). \tag{2}$$

The verifier is usually correct but deliberately too weak to override an already overconfident wrong policy. This does not force confirmation bias: it creates the condition under which bias can either grow or be corrected.

3.1 Probing confirmation bias

Because the simulator hides an oracle action from the learner, we can diagnose pseudo-targets without changing training. We report:

$$\mathrm{PseudoErr}_k = \Pr[\hat{a}_k \neq a^\star], \tag{3}$$

$$\mathrm{SelfReinforce}_k = \Pr[\arg\max_a \pi_k(a \mid s) \neq a^\star \ \wedge\ \hat{a}_k = \arg\max_a \pi_k(a \mid s)], \tag{4}$$

$$\mathrm{Amplification} = \mathrm{PseudoErr}_K - \mathrm{PseudoErr}_1. \tag{5}$$

We additionally track normalized pseudo-action entropy. Falling entropy together with rising self-reinforced error distinguishes collapse from merely hard data.

In plain language, pseudo-label error asks "was the training answer wrong?" and self-reinforcement asks the stricter question "did the agent copy its own wrong answer into the training set?" For example, if the correct tool is `search`, the policy chooses `click`, and the constructed target is also `click`, the state counts toward both metrics. If the verifier repairs the target to `search`, it counts toward neither.

3.2 Probing pattern loss

Workflows arrive in non-IID waves. For pattern p, let k_p be its first exposure and $S_{p,k}$ its full-episode success after update k. We define

$$\textsc{PatternLoss}(p) = \max_{k \geq k_p} S_{p,k} - S_{p,K}. \tag{6}$$

This asks whether the agent *previously knew* a workflow and lost it after later self-evolution. Frozen has zero pattern loss mechanically because it never updates; it is an ability anchor, not a continual-learning solution.

Table 1: The two pitfalls are measured at different levels. Oracle actions are available to the evaluator only; training receives policy probabilities and weak verifier feedback.

Metric	What is counted	What a high value means
Pseudo-label error	$\hat{a} \neq a^\star$, averaged over collected decision states	The agent is manufacturing incorrect supervision.
Self-reinforced error	The greedy policy action is wrong and the pseudo-label copies that exact action	The model's own error is directly fed back as a target.
Pattern loss	For each workflow, best post-exposure episode success minus final success	A workflow was acquired and later erased; this is not merely failure to learn.

4 Lazy Distillation

LAZYDISTILL separates collection time from internalization time. The method has two deliberately simple modules: a *frozen collection window* that blocks fast error feedback and a *pooled update* that jointly rehearses every pattern observed in that window. For a window of W non-IID waves, it holds the collector fixed at $\bar{\pi}_j$:

$$\mathcal{B}_j = \bigcup_{w=1}^{W} \widehat{\mathcal{D}}_{j,w}(\bar{\pi}_j). \tag{7}$$

Only after all waves arrive does it update on the pooled buffer:

$$\pi_{j+1} = \arg\min_{\pi} \sum_{x \in \mathcal{B}_j} \ell(\pi(\cdot \mid s_x), \hat{a}_x)\,. \tag{8}$$

LazyDistill in four steps

1. Snapshot the current agent as the frozen collector $\bar{\pi}_j$.
2. Collect W non-IID waves with the same $\bar{\pi}_j$; do not update between waves.
3. Construct pseudo-targets with policy probabilities and weak verifier feedback, then pool all waves into $\mathcal{B}_j$.
4. Train once on $\mathcal{B}_j$ to obtain π_{j+1} and begin the next window.

Why it targets confirmation bias. Within a window, an early wrong update cannot change later trajectories because there is no within-window update. The loop gain of error→parameter→data is therefore reduced from every wave to once per window.

Why it targets pattern loss. Common and rare waves are co-present in the same optimization event. Their gradients compete jointly instead of the final dominant wave overwriting all earlier waves. This is different from replay: replay repairs a sequential update after it occurs and may repeatedly replay corrupted pseudo-labels.

5 Experimental Setup

5.1 Dataset and environment

We generate 24 procedural workflow patterns organized into 12 confusable common/rare pairs. Pair members have highly similar observation cues but disagree on half of their hidden action programs. Each observation includes the workflow cue, current progress, previous tool call, and success/failure feedback. Later cycles oversample common patterns, creating a realistic dominant-trajectory stream without deleting minority patterns.

The main setting uses $H = 8$, three attempts per stage, six waves, 12 evolution cycles, and five seeds. The scale setting uses $H = 16$, 24 evolution cycles, and three seeds. Every method starts from the same warm-start policy and sees the same schedule. Results report mean and sample standard deviation.

Table 2: Controlled dataset and protocol. The scale setting changes only the two axes central to our hypothesis: episode horizon and number of self-evolution cycles.

Property	Main	Scale	Purpose
Workflow patterns	24	24	12 common/rare confusable pairs
Actions (tools)	8	8	Discrete action choice at each state
Dependent steps per episode	8	16	A wrong step changes later observations
Attempts allowed per step	3	3	Environment feedback permits local recovery
Non-IID waves per cycle	6	6	Different pattern subsets arrive sequentially
Self-evolution cycles	12	24	Repeated collect–label–update loops
Training tasks per active pattern	24	24	Identical data budget across methods
Evaluation tasks per pattern	32	32	Full evaluation of all 24 patterns
Random seeds	5	3	Mean and sample standard deviation

What one episode looks like. A workflow is a hidden length-H program over eight tool actions. At every step, the policy observes a 24-dimensional workflow cue, current program position, previous action, and success/failure feedback. Confusable pair members share similar cues but disagree on half of their correct actions. Thus memorizing the dominant member harms its rare partner. The full episode succeeds only if the agent completes all H dependent stages within three attempts per stage, making the scale setting genuinely long-horizon rather than 16 independent classification examples.

How non-IID evolution is induced. Each cycle contains six waves, and each wave activates a subset of patterns. Later cycles oversample common members while rare members appear less often. All methods receive the exact same wave order and warm-start state. This schedule creates the two necessary stressors without hard-coding failure: self-generated labels can recursively change, and sequentially dominant patterns can overwrite rare ones.

5.2 Comparison methods

- **Frozen**: no parameter updates.
- **Eager self-distillation**: update after every wave.
- **Confidence-filtered eager**: retain pseudo-labels above a fixed confidence threshold, following pseudo-label selection defenses [Radhakrishnan et al., 2023, Rodemann et al., 2023].
- **Mean Teacher**: use an exponential moving-average teacher [Tarvainen and Valpola, 2017].
- **Eager + replay**: mix each current wave with historical trajectories, following continual-learning replay [Yoo et al., 2024] and growing-batch ReST [Gulcehre et al., 2023].
- **SDAR-style gated eager**: adapt SDAR's detached positive-gap gate [Lu et al., 2026] to our action-level policy. For pseudo-target $\hat{a}$, $g = \sigma(\beta[\log q(\hat{a} \mid s) - \log \pi(\hat{a} \mid s)])$ is detached and multiplies cross-entropy. This isolates SDAR's gating mechanism but is *not* an exact reproduction of its token-level GRPO system.
- **Lazy**: freeze collection for six waves and then update on all collected pseudo-labels.

These implementations isolate the papers' relevant mechanisms; they are not claimed as exact reproductions on the original datasets.

Table 3: Eight-step, 12-cycle results over five seeds (mean ± s.d.). Arrows indicate the preferred direction.

Method	Success ↑	Pseudo error ↓	Self-reinforce ↓	Pattern loss ↓
Frozen	35.8 ± 5.9	46.4 ± 7.1	44.9 ± 7.2	0.0 ± 0.0
Eager	0.0 ± 0.0	94.4 ± 1.2	94.3 ± 1.2	39.6 ± 9.7
Confident eager	54.0 ± 30.2	56.8 ± 17.2	56.5 ± 17.2	26.8 ± 14.4
Mean Teacher	62.8 ± 12.9	49.0 ± 8.1	48.2 ± 8.2	24.3 ± 9.3
Eager + replay	56.6 ± 9.3	60.4 ± 8.1	60.2 ± 8.1	10.5 ± 2.6
Lazy (timing)	**74.0 ± 17.8**	**43.2 ± 12.5**	**43.0 ± 12.5**	**3.6 ± 3.1**

Table 4: Sixteen-step, 24-cycle long-horizon scale experiment.

Method	Success ↑	Pseudo error ↓	Self-reinforce ↓	Pattern loss ↓
Frozen	52.9 ± 3.5	24.0 ± 3.5	21.9 ± 3.5	0.0 ± 0.0
Eager	0.0 ± 0.0	95.3 ± 2.5	95.2 ± 2.6	19.6 ± 8.6
Confident eager	0.0 ± 0.0	93.6 ± 1.7	93.5 ± 1.7	12.2 ± 3.9
Mean Teacher	79.1 ± 9.6	30.3 ± 9.1	29.8 ± 9.1	11.3 ± 4.4
Eager + replay	24.3 ± 4.0	69.4 ± 1.6	69.3 ± 1.6	10.9 ± 7.6
SDAR-style gate	3.3 ± 3.0	88.3 ± 4.0	88.0 ± 4.1	41.0 ± 15.4
Lazy	**79.6 ± 0.8**	**29.9 ± 2.9**	**29.8 ± 2.9**	**6.4 ± 4.9**

6 Preliminary Results

Evaluation logic

Q1 Does repeated eager self-distillation actually create confirmation bias and pattern loss?

Q2 Do target filtering, teacher smoothing, replay, or delayed timing resolve both failures?

Q3 Does the distinction persist when both episode horizon and evolution depth are doubled?

6.1 Does the long self-evolution failure exist?

Table 3 first establishes the phenomenon. Eager self-distillation collapses from a nonzero frozen policy to zero full-episode success. Its pseudo-label error reaches 94.4% on average, and nearly every wrong greedy action becomes its own target. Its pseudo-action entropy also falls, consistent with mode collapse rather than uniformly hard tasks.

Takeaway. The first experiment establishes the phenomenon before evaluating the cure: eager learning converts a usable 35.8% frozen policy into a 0% policy, and 94.3% of collected states directly reinforce the policy's own wrong action.

6.2 Probing whether lazy learning addresses both failures

The comparison separates three partial defenses. Confidence filtering is high-variance because the evolving policy can become confidently wrong. Mean Teacher stabilizes targets but still performs sequential non-IID updates. Replay sharply reduces pattern loss but retains high pseudo-label error because old incorrect targets are replayed. Lazy timing is the only updating method in Table 3 with both the lowest error feedback and near-zero pattern loss. A separate probe found that naively adding a confidence gate to Lazy can hurt at the longest horizon because the evolving policy becomes confidently wrong. This negative result strengthens the timing interpretation: confidence selection is not a substitute for breaking the recursive update loop.

6.3 Scale with horizon and evolution depth

The decisive scale experiment increases both axes that generate the recursive failure: horizon from 8 to 16 and evolution cycles from 12 to 24. The final three-seed values are shown in Table 4.

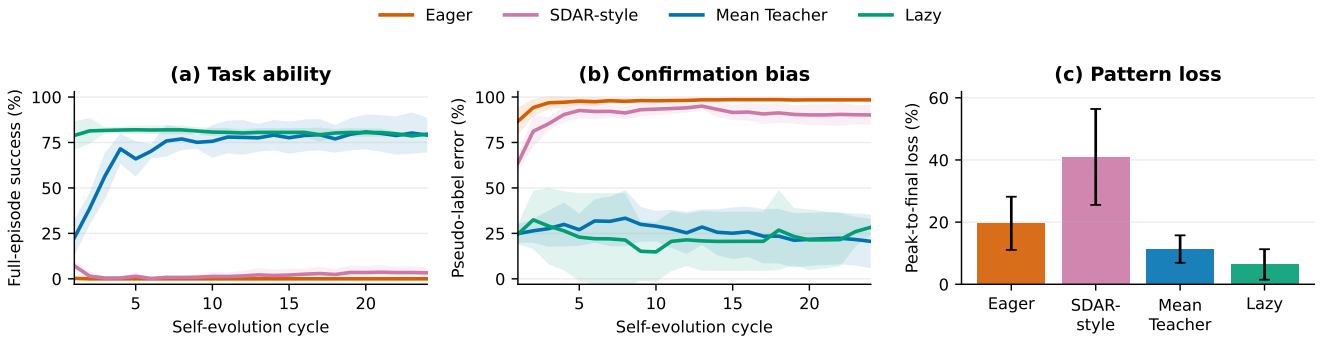


Figure 2: Failures emerge over repeated loops, not from one difficult batch. Mean ± one standard deviation across three seeds in the 16-step, 24-cycle setting. Eager and SDAR-style learning rapidly approach near-total pseudo-label error and lose task ability. Lazy remains stable and has the lowest pattern loss among updating methods.

Takeaway. Difficulty amplifies the distinction. At twice the horizon and twice the evolution depth, Lazy and Mean Teacher preserve task ability, but Lazy retains workflows better; confidence gates, replay, and SDAR-style eager gating do not jointly control both failure metrics.

6.4 Why SDAR-style gating does not replace lazy timing

The SDAR-style adaptation reaches only $3.3\% \pm 3.0\%$ success and accumulates $88.3\% \pm 4.0\%$ pseudo-label error. This negative result should not be read as a leaderboard claim against full SDAR: our controlled policy has neither language tokens nor GRPO rewards nor a privileged skill-context teacher. Instead, it is a causal probe of the transferable mechanism. A positive-gap gate controls *how strongly* the learner fits each current target, but eager updates still change the collector after every wave. Once a wrong pattern becomes confident, its target can retain substantial weight, affect later state visitation, and overwrite earlier patterns.

Lazy changes the missing axis: *when* learning is allowed to affect data collection. Freezing the collector blocks within-window error→data feedback, and pooling all waves makes minority and majority patterns co-present. Thus SDAR-style gating and Lazy are conceptually orthogonal; a full Lazy-SDAR experiment on ALFWorld or WebShop is an important next step.

7 Limitations and Next Steps

This is a procedural stress test with a lightweight policy and a simulator oracle. It is designed for causal probing, not external validity. The hyperparameter regime was calibrated on one seed before multi-seed evaluation. The replay baseline receives additional historical examples, so it is data-reuse matched rather than optimizer-step matched. Our next experiment should preserve the same frozen-window intervention in ALFWorld, WebShop, or a comparable language-agent environment. The SDAR row is a mechanism adaptation rather than an exact reproduction; a fair external comparison requires the authors' GRPO stack, language-model teacher, and original ALFWorld/SearchQA/WebShop protocols.

8 Conclusion

Long-horizon self-evolution turns ordinary pseudo-label noise into a dynamical system: the learner changes the data that will supervise its next version. Preliminary results show that eager updates can simultaneously amplify wrong actions and erase old workflows. Lazy collection attacks the causal link responsible for the first failure and pooled optimization attacks the sequential overwrite responsible for the second. Naive confidence gating is not sufficient when confidence itself is corrupted.

References

Renat Aksitov, Sobhan Miryoosefi, Zonglin Li, et al. Rest meets react: Self-improvement for multi-step reasoning llm agent. *arXiv preprint arXiv:2312.10003*, 2023.

Eric Arazo, Diego Ortego, Paul Albert, Noel E. O'Connor, and Kevin McGuinness. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *International Joint Conference on Neural Networks*, 2020.

Jingwen Chen, Wenkai Yang, Shengda Fan, et al. Rethinking continual experience internalization for self-evolving llm agents. *arXiv preprint arXiv:2606.04703*, 2026.

Zi-Yi Dou, Cheng-Fu Yang, Xueqing Wu, Kai-Wei Chang, and Nanyun Peng. Re-rest: Reflection-reinforced self-training for language agents. *arXiv preprint arXiv:2406.01495*, 2024.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Self-distilled agentic reinforcement learning. *arXiv preprint arXiv:2605.15155*, 2026.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*, 2023.

Aswathnarayan Radhakrishnan, Jim Davis, Zachary Rabin, Benjamin Lewis, Matthew Scherreik, and Roman Ilin. Enhancing self-training methods. *arXiv preprint arXiv:2301.07294*, 2023.

Julian Rodemann, Jann Goschenhofer, Emilio Dorigatti, Thomas Nagler, and Thomas Augustin. Approximately bayes-optimal pseudo-label selection. In *Uncertainty in Artificial Intelligence*, 2023.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems*, 2017.

Jinyang Wu, Shuo Yang, Zhengxi Lu, Fan Zhang, Yuhao Shen, Lang Feng, Haoran Luo, Zheng Lian, Shuai Zhang, Zhengqi Wen, and Jianhua Tao. Seed: Self-evolving on-policy distillation for agentic reinforcement learning. *arXiv preprint arXiv:2607.14777*, 2026.

Shuo Yang, Jinyang Wu, Zhengxi Lu, Yuhao Shen, Fan Zhang, Lang Feng, Shuai Zhang, Haoran Luo, Zheng Lian, Zhengqi Wen, and Jianhua Tao. Opid: On-policy skill distillation for agentic reinforcement learning. *arXiv preprint arXiv:2606.26790*, 2026.

Jinsoo Yoo, Yunpeng Liu, Frank Wood, and Geoff Pleiss. Layerwise proximal replay: A proximal point method for online continual learning. In *International Conference on Machine Learning*, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. *arXiv preprint arXiv:2203.14465*, 2022.